

LAPORAN TUGAS BESAR
JARINGAN SENSOR NIRKABEL
ANT COLONY DAN SHORTEST PATH ALGORITHM



DISUSUN OLEH KELOMPOK 3

ANGGOTA :

- 1. DINDA MESTIKA (2111511006)**
- 2. ARISTA ZHAFIRAH (2111511022)**
- 3. TEGUH ALAM NASHRAH (2111512026)**
- 4. OMAR FAUZI RAHMAN (2111513002)**
- 5. JEVON AL SALGUS (2111513018)**

DOSEN PENGAMPU :

DODY ICHWANA PUTRA, PhD

NIP. 198611072015041001

DEPARTEMEN TEKNIK KOMPUTER
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2024

BAB I

PENDAHULUAN

1.1. Algoritma *Ant Colony*

a. Definisi

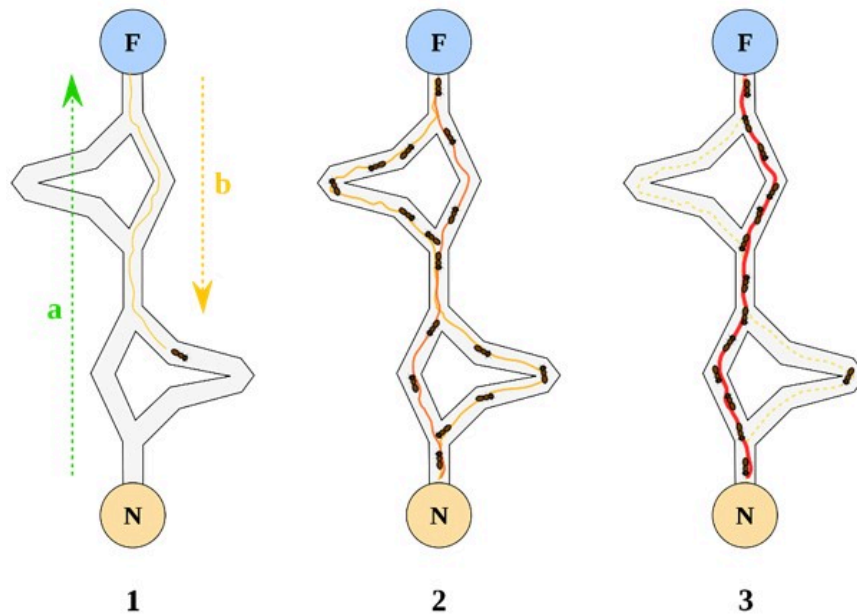
Ant-Colony Optimization (ACO) termasuk dalam kelompok *Swarm Intelligence*, yang merupakan sebuah paradigma yang digunakan untuk menyelesaikan masalah optimasi dengan inspirasi yang diambil dari perilaku kawanan serangga. ACO adalah teknik berbasis probabilitas untuk menyelesaikan masalah, yang terinspirasi dari perilaku semut dalam koloni saat mencari makanan. ACO biasanya diterapkan untuk menyelesaikan masalah optimasi diskrit dan masalah kompleks dengan banyak variabel. Meskipun hasil yang diperoleh dengan ACO mungkin tidak sepenuhnya optimal, namun mendekati optimal.

Swarm intelligence adalah sebuah pendekatan *relative* baru untuk pemecahan masalah yang mengambil inspirasi dari social perilaku serangga dan hewan lainnya. *Ant Colony Optimization* (ACO) adalah salah satu algoritma yang paling sukses dalam bidang *swarm intelligence*.

b. Cara Kerja

Salah satu hal menarik dari perilaku semut adalah kemampuannya dalam menemukan jarak terpendek antara sarang mereka dan sumber makanan. Setiap semut dalam kawanan yang bergerak akan meninggalkan *pheromone* (zat kimia) di jalur yang dilaluinya. *Pheromone* ini berfungsi sebagai sinyal bagi semut-semut lainnya.

Pheromone adalah zat kimia yang berasal dari kelenjar endokrin dan digunakan oleh makhluk hidup untuk mengenali sesama jenis, individu lain, kelompok, dan untuk membantu dalam proses reproduksi. Berbeda dengan hormon, *pheromone* menyebar ke luar tubuh dan hanya dapat dikenali serta mempengaruhi individu lain yang sejenis (satu spesies).



Gambar 1.1 Ilustrasi Algoritma *Ant Colony*

Jalur yang pendek akan menyisakan sinyal yang lebih kuat. Semut berikutnya, pada saat memutuskan jalur mana yang harus dipilih, biasanya akan cenderung memilih untuk mengikuti jalur dengan sinyal yang paling kuat, sehingga jalur terpendek akan ditemui karena lebih banyak semut yang akan melewati jalur tersebut. Semakin banyak semut yang menempuh suatu lintasan tertentu, maka aroma *pheromone* pada lintasan tersebut akan semakin kuat.

Pada awalnya semut akan terdistribusi merata melalui dua lintasan. Setelah beberapa saat semut-semut mulai memilih lintasan terpendek karena *pheromone* yang ditinggalkan semut akan mengumpul di lintasan yang lebih pendek.

c. Implementasi dalam Jaringan Sensor Nirkabel

Implementasi *Ant Colony Optimization* (ACO) pada konsep *Wireless Sensor Network* (WSN) bertujuan untuk mengoptimalkan beberapa aspek seperti *routing* (pengaturan jalur pengiriman data), penentuan lokasi sensor, dan manajemen energi.

1. *Routing* dalam WSN Menggunakan ACO :

Routing adalah proses penentuan jalur yang akan dilalui data dari sensor menuju ke *sink* (penerima data). ACO dapat digunakan untuk menemukan

route optimal yang meminimalkan konsumsi energi dan memperpanjang umur jaringan.

Langkah-langkah Implementasi:

- **Initialization** : Setiap *node* sensor diinisialisasi dengan jumlah *pheromone* awal yang sama.
- **Ant Deployment** : Semut virtual dilepaskan dari *node* sumber untuk mencari jalur ke *node* tujuan (*sink*).
- **Path Selection** : Semut memilih jalur berdasarkan probabilitas yang dihitung dari konsentrasi *pheromone* dan faktor jarak. Semut cenderung memilih jalur dengan *pheromone* lebih tinggi tetapi juga mempertimbangkan jarak untuk menghindari *route* yang terlalu panjang.
- **Pheromone Update** : Setelah semut mencapai tujuan, jalur yang dilalui diperbarui dengan peningkatan konsentrasi *pheromone* berdasarkan kualitas jalur (misalnya, *route* lebih pendek atau konsumsi energi lebih rendah). *Pheromone* pada jalur yang tidak digunakan akan menguap seiring waktu.
- **Iteration** : Proses ini diulang beberapa kali untuk menemukan jalur optimal yang meminimalkan konsumsi energi dan *delay*.

2. Manajemen Energi dengan ACO :

Manajemen energi penting dalam WSN untuk memperpanjang masa pakai jaringan karena sensor biasanya memiliki sumber daya energi yang terbatas.

Langkah-langkah Implementasi :

- **Initialization** : Inisialisasi energi awal untuk setiap *node* dan konsentrasi *pheromone*.
- **Ant Deployment** : Semut virtual dilepaskan untuk menemukan jalur komunikasi yang meminimalkan konsumsi energi.
- **Path Selection** : Semut memilih jalur berdasarkan energi residual dan konsentrasi *pheromone*. Jalur yang menghabiskan energi lebih sedikit akan lebih disukai.
- **Pheromone Update** : Jalur yang dipilih dengan konsumsi energi rendah akan diperkuat dengan *pheromone*, sedangkan jalur lainnya akan mengalami penguapan *pheromone*.

- **Iteration** : Proses ini diulang untuk memastikan jalur komunikasi yang dipilih selalu mengutamakan efisiensi energi.

d. Kelebihan Implementasi ACO pada Konsep WSN

1. Distribusi Beban :

Mengurangi kemacetan pada *node* tertentu dengan mendistribusikan beban pengiriman data secara merata.

2. Efisiensi Energi :

Meminimalkan konsumsi energi sehingga memperpanjang umur baterai sensor.

3. Ketahanan :

Adaptif terhadap perubahan topologi jaringan dan kegagalan *node*.

4. Optimalisasi Cakupan :

Penempatan sensor yang optimal untuk cakupan area maksimal dengan jumlah sensor minimal.

1.2. Algoritma *Shortest Path*

a. Definisi

Shortest Path Algorithm (SPA) merupakan metode pencarian lintasan terpendek pada sebuah graf bersambung, berarah dan berantai. Untuk pencarian lintasan terpendek dari sebuah graf bersambung dan berarah, dapat digunakan perhitungan cara manual atau dengan melalui komputer.

Algoritma *Shortest Path* adalah algoritma yang digunakan untuk menemukan jalur terpendek antara dua *node* dalam sebuah graf. Ketika setiap *node* tersebut dapat mewakili berbagai entitas seperti lokasi, titik jaringan, atau lainnya, sementara *edge*-nya (sisi) mewakili koneksi antara *node* tersebut dengan bobot tertentu yang menunjukkan jarak, waktu, atau biaya.

b. Cara Kerja

Beberapa algoritma yang terkenal untuk menemukan jalur terpendek adalah:

1. *Dijkstra's Algorithm* :

Inisialisasi : Setiap *node* diberi jarak awal tak terbatas kecuali *node* sumber yang diberi jarak 0

Pemrosesan *Node* : *Node* dengan jarak terkecil diproses pertama kali. Jarak ke *node* tetangga diperbarui jika melalui *node* saat ini memberikan jalur yang lebih pendek.

Iterasi : *Node* yang sudah diproses dipindahkan dari daftar yang belum diproses hingga semua *node* diproses atau *node* tujuan ditemukan.

2. ***Bellman-Ford Algorithm :***

Inisialisasi : Sama dengan Dijkstra, semua jarak diinisialisasi tak terbatas kecuali sumber.

Relaksasi *Edge* : Setiap *edge* dalam graf di relaksasi untuk memperbarui jarak terpendek. Proses ini diulang hingga $|V| - 1$ kali (dengan $|V|$ adalah jumlah *node*).

Deteksi Siklus Negatif : Algoritma dapat mendeteksi adanya siklus negatif dalam graf jika ditemukan relaksasi tambahan setelah $|V| - 1$ iterasi.

3. ***Floyd-Warshall Algorithm :***

Inisialisasi : Matriks jarak diinisialisasi dengan jarak langsung antara *node*.

Iterasi : Untuk setiap pasangan *node*, diperiksa apakah jalur melalui *node* perantara memberikan jarak lebih pendek, dan matriks diperbarui jika ditemukan jalur yang lebih pendek.

c. **Implementasi**

Dalam mengimplementasikan algoritma *route* terpendek pada konsep jaringan sensor nirkabel, harus memperhatikan beberapa hal, diantaranya :

1. ***Graph Representation :***

WSN diwakili sebagai graf dengan *node* sebagai sensor dan *edge* sebagai koneksi antar sensor dengan bobot tertentu yang menggambarkan jarak atau biaya energi.

2. ***Node Initialization :***

Setiap *node* diinisialisasi dengan jarak tak terbatas kecuali *node* sumber yang diberi jarak 0 (jika menggunakan Dijkstra atau Bellman-Ford).

3. *Path Calculation* :

Algoritma *Shortest Path* dijalankan untuk menghitung jalur terpendek dari *node* sumber ke *node* tujuan atau semua *node* lainnya (bergantung pada kebutuhan aplikasi).

4. *Routing Table Update* :

Setiap *node* memperbarui tabel *routing* berdasarkan hasil algoritma sehingga data dapat dikirim melalui jalur terpendek yang teridentifikasi.

5. *Data Transmission* :

Data dikirim dari *node* sumber ke *node* tujuan menggunakan jalur terpendek yang ditentukan oleh algoritma.

d. Perbedaan dengan *Ant Colony*

Algoritma *ant colony* dan algoritma *shortest path* memiliki beberapa perbedaan dalam hal cara kerja, kompleksitas, dan lainnya. Berikut penjelasannya :

1. Cara Kerja :

Shortest Path Algorithms : Menggunakan pendekatan deterministik untuk menemukan jalur terpendek secara langsung (misalnya, Dijkstra menghitung jarak minimum secara iteratif)

ACO : Menggunakan agen (semut) yang mencari jalur secara iteratif dan probabilistik berdasarkan konsentrasi *pheromone* yang diperbarui.

2. Fungsionalitas :

Shortest Path Algorithms : Dirancang untuk menemukan jalur terpendek dalam graf statis dengan bobot tetap.

ACO : Dirancang untuk menemukan jalur yang mendekati optimal dalam graf yang mungkin berubah-ubah, menggunakan mekanisme pembaruan *pheromone* untuk adaptasi

3. Kualitas :

Shortest Path Algorithms : Memberikan jalur terpendek yang optimal secara matematis

ACO : Menghasilkan solusi mendekati optimal, dengan kemungkinan variasi tergantung pada jumlah iterasi dan parameter algoritma.

4. Performa :

Shortest Path Algorithms : Biasanya lebih cepat dan efisien dalam graf statis, dengan waktu komputasi yang dapat diprediksi.

ACO : Lebih lambat karena melibatkan banyak iterasi dan agen (semut), tetapi mampu menemukan solusi yang baik dalam masalah optimasi yang kompleks.

5. Penggunaan Energi :

Shortest Path Algorithms : Efisien dalam penggunaan energi ketika diterapkan pada jaringan statis dengan komunikasi yang minimal.

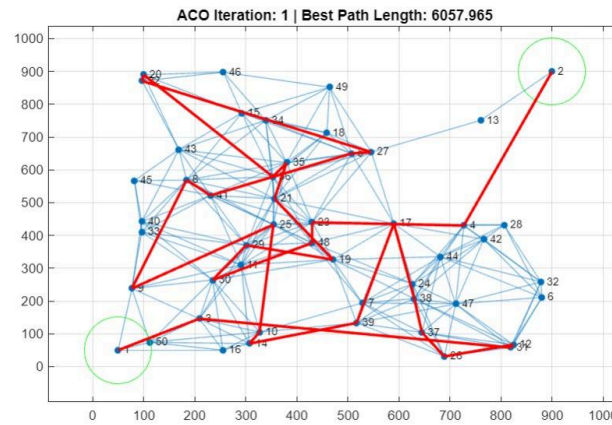
ACO : Dapat menggunakan lebih banyak energi karena iterasi dan komunikasi berulang, tetapi mampu mengoptimalkan penggunaan energi dalam jangka panjang melalui adaptasi dan pembaruan jalur.

BAB II

HASIL PERCOBAAN

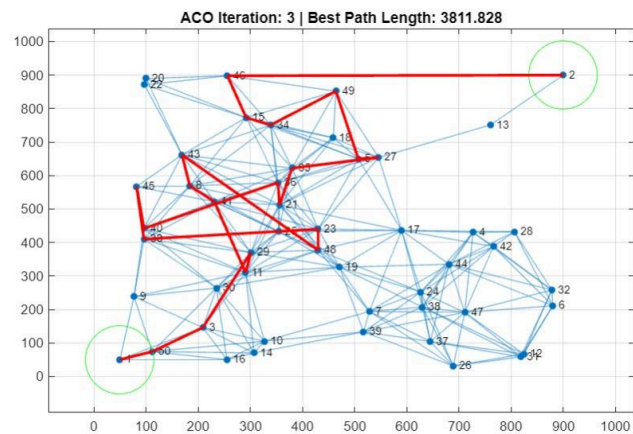
2.1. Percobaan *Ant Colony*

a. Iterasi ke-1



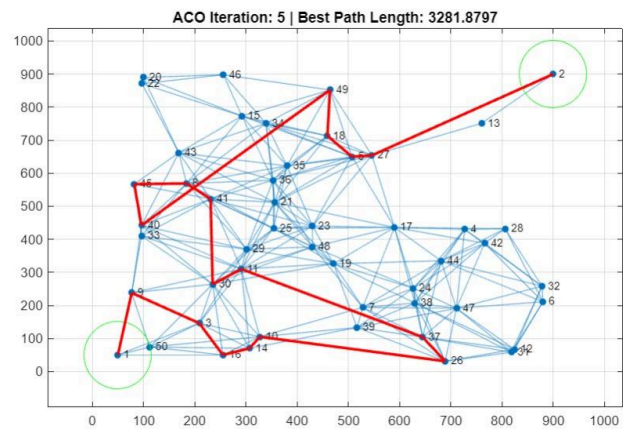
Gambar 2.1 Iterasi ke-1 Percobaan *Ant Colony*

b. Iterasi ke-3



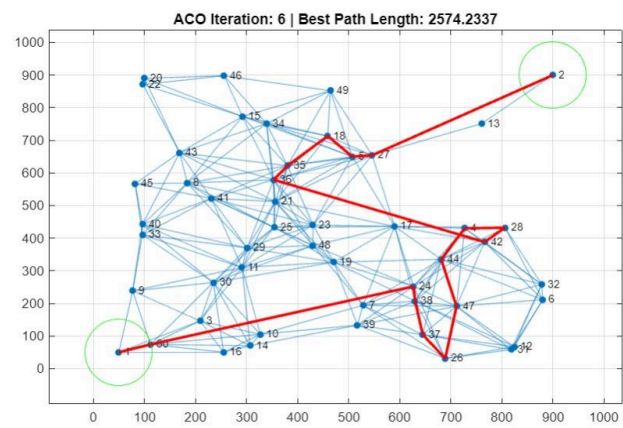
Gambar 2.2 Iterasi ke-3 Percobaan *Ant Colony*

c. Iterasi ke-5



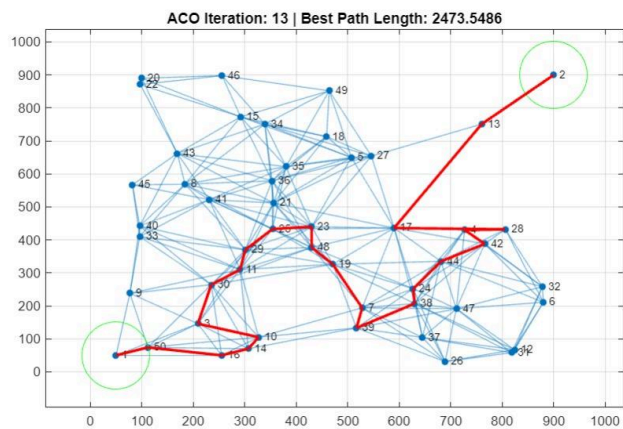
Gambar 2.3 Iterasi ke-5 Percobaan *Ant Colony*

d. Iterasi ke-6



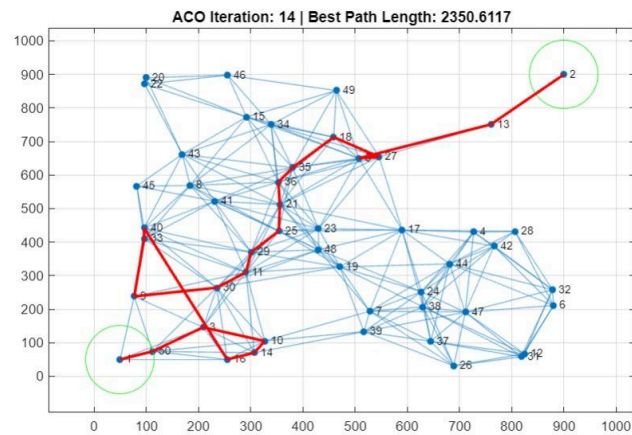
Gambar 2.4 Iterasi ke-6 Percobaan *Ant Colony*

e. Iterasi ke-13



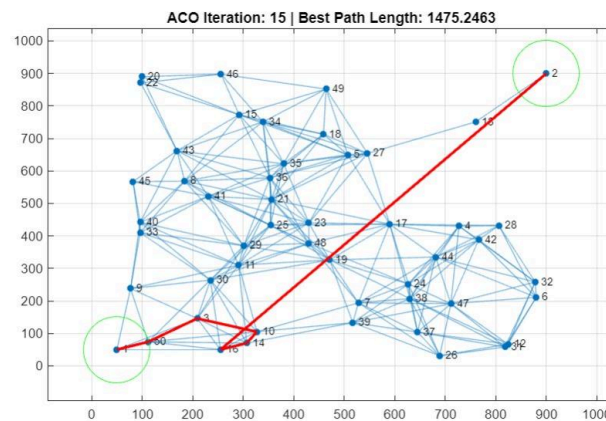
Gambar 2.5 Iterasi ke-13 Percobaan *Ant Colony*

f. Iterasi ke-14



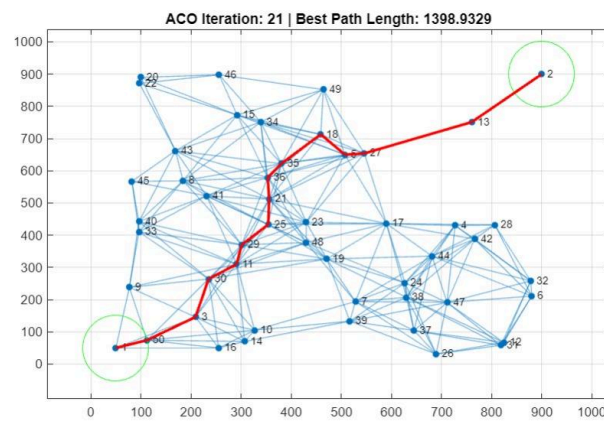
Gambar 2.6 Iterasi ke-14 Percobaan *Ant Colony*

g. Iterasi ke-15



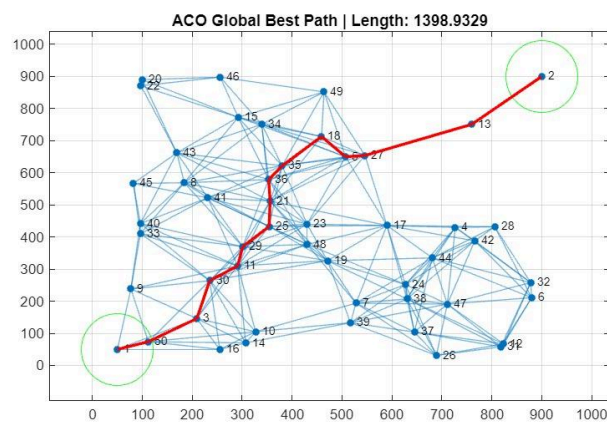
Gambar 2.7 Iterasi ke-15 Percobaan *Ant Colony*

h. Iterasi ke-21



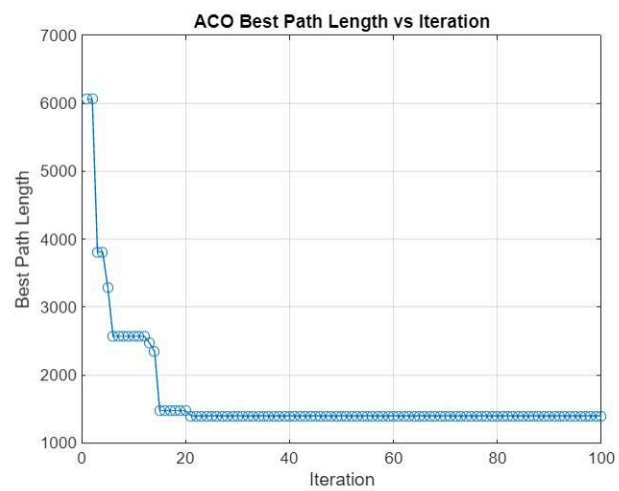
Gambar 2.8 Iterasi ke-21 Percobaan *Ant Colony*

i. Best Path



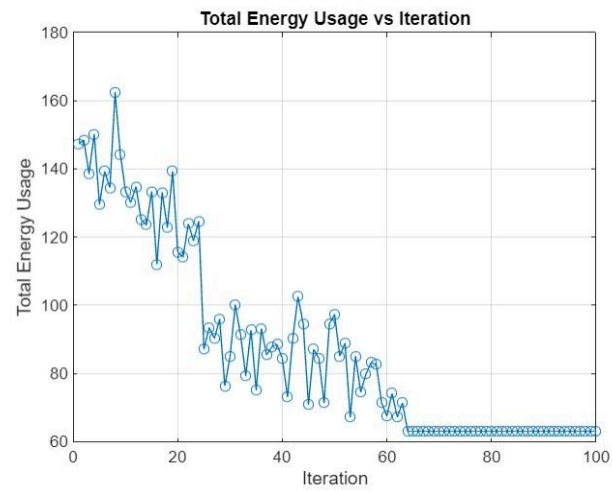
Gambar 2.9 Best Path Percobaan *Ant Colony*

j. Best Path vs Iteration



Gambar 2.10 *Best Path vs Iteration* Percobaan *Ant Colony*

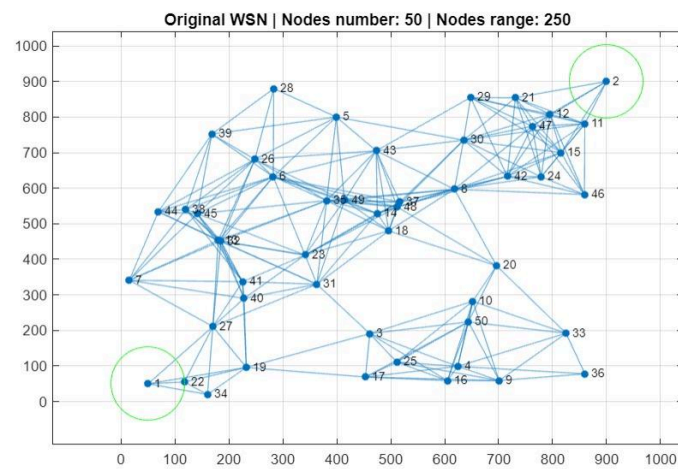
k. Total Energy Used vs Iteration



Gambar 2.11 *Total Energy Used vs Iteration* Percobaan *Ant Colony*

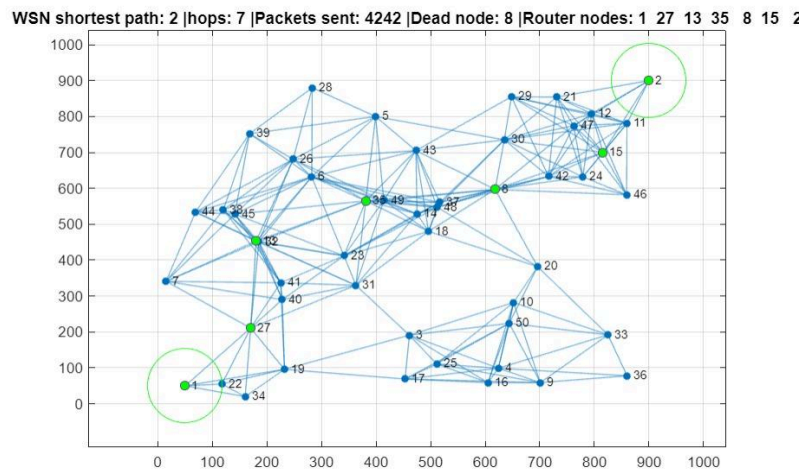
2.2. Percobaan *Shortest Path*

a. Iterasi ke-1



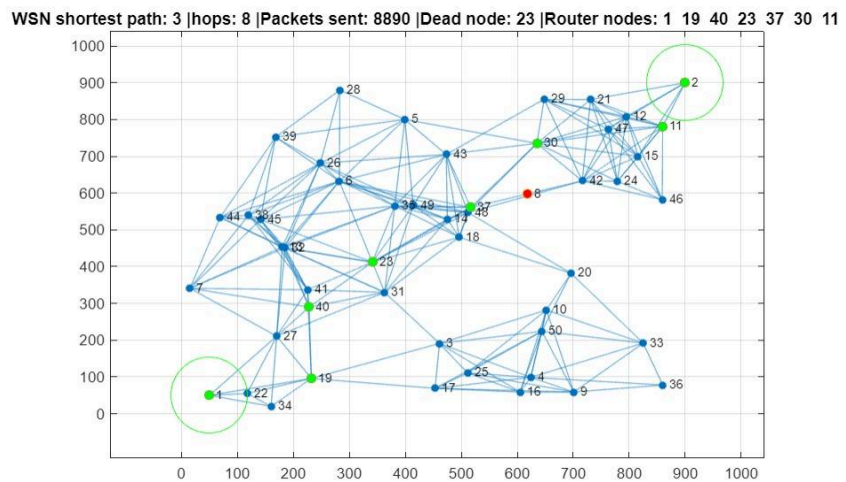
Gambar 2.12 Iterasi ke-1 Percobaan *Shortest Path*

b. Iterasi ke-2



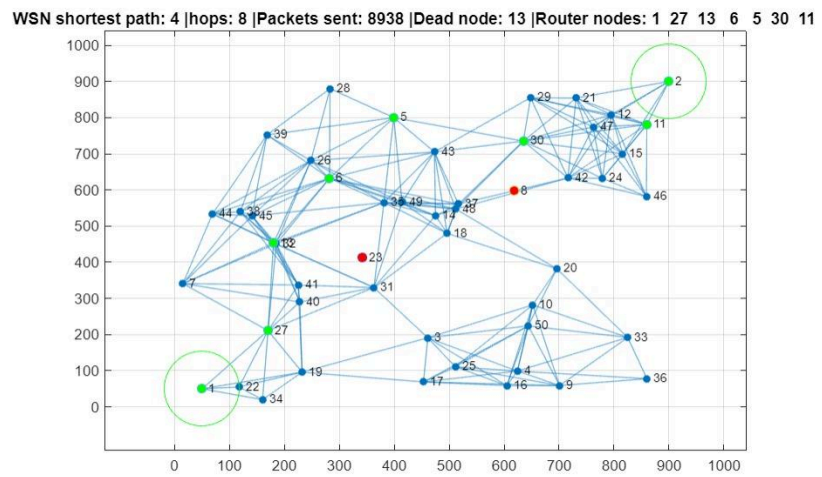
Gambar 2.13 Iterasi ke-2 Percobaan *Shortest Path*

c. Iterasi ke-3



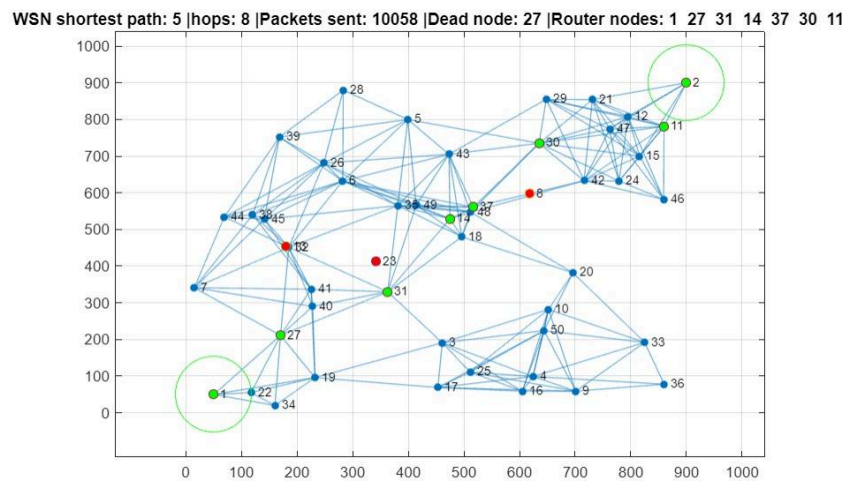
Gambar 2.14 Iterasi ke-3 Percobaan *Shortest Path*

d. Iterasi ke-4



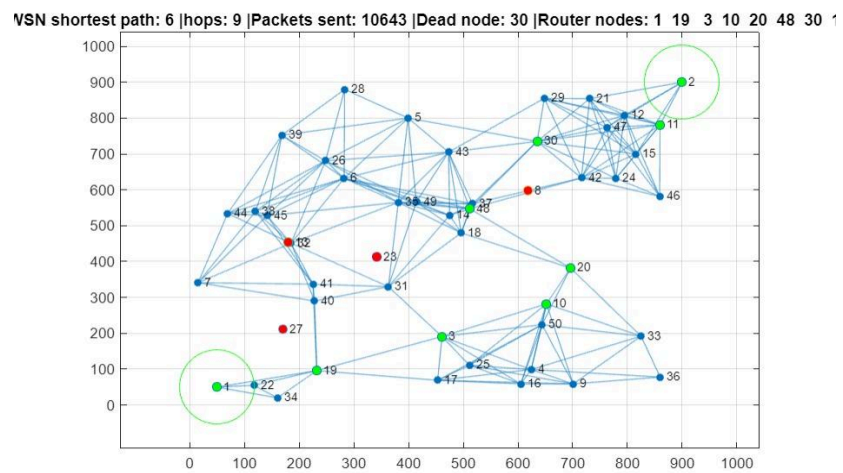
Gambar 2.15 Iterasi ke-4 Percobaan *Shortest Path*

e. Iterasi ke-5



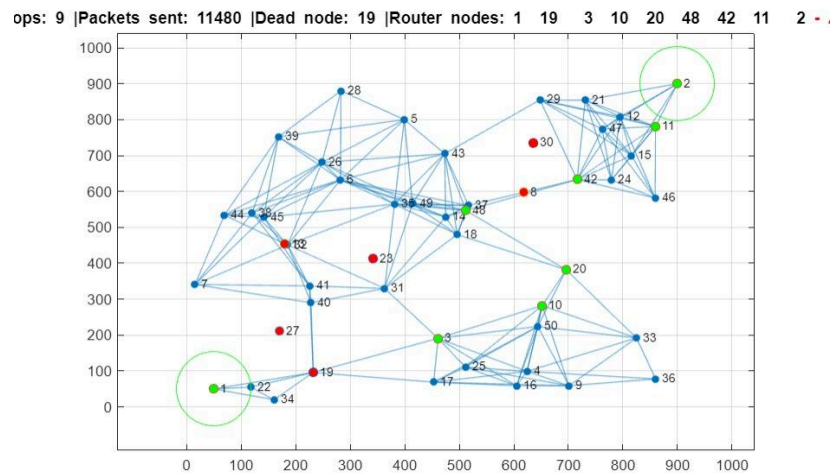
Gambar 2.16 Iterasi ke-5 Percobaan *Shortest Path*

f. Iterasi ke-6



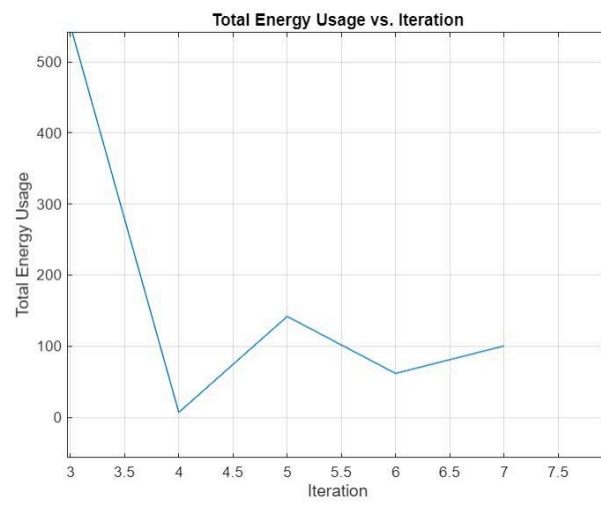
Gambar 2.16 Iterasi ke-6 Percobaan *Shortest Path*

g. Iterasi ke-7



Gambar 2.17 Iterasi ke-7 Percobaan *Shortest Path*

h. Total Energy Usage vs Iteration



Gambar 2.18 *Total Energy Usage vs Iteration Percobaan Shortest Path*

BAB III

ANALISA

3.1. *Ant Colony*

Pada percobaan yang dilakukan, digunakan jumlah total *node* sebanyak 50 *node* dan dihasilkan iterasi atau perulangan sebanyak 100 kali hingga *best path* atau jalur ideal terpendek untuk transmisi data ditemukan. Proses transmisi data pada percobaan yang dilakukan dimulai dari *node* 1 dan *node* 2 sebagai destinasi transmisi data, tetapi untuk mencapai *node* 2 memerlukan beberapa iterasi untuk mendapatkan jalur terbaik dengan waktu sesingkat mungkin. Dalam setiap iterasi yang terjadi, jumlah *node* yang terlibat dalam transmisi data akan terus mengalami penurunan hingga *best path* ditemukan.

Adapun energi awal setiap *node* diacak dalam rentang energi awal antara 80mW hingga 100mW. Hal tersebut ditandai dengan penggunaan fungsi '**dataset.minEnergy = 80**' dan '**dataset.maxEnergy = 100**'. Sehingga, jika konsumsi energi berada di luar rentang tersebut dapat mengakibatkan *node* mati karena kehabisan energi. Dari hasil percobaan yang dilakukan, didapatkan *output* penggunaan energi yang memenuhi setiap rentang tersebut untuk setiap *node*-nya, sehingga tidak ada terjadinya masa kehabisan energi selama proses transmisi data.

Penentuan *best path* atau jalur terbaik dengan metode ACO ini menerapkan fungsi probabilitas $P_i = (R_i / (R_1 + R_2))$, dimana R itu merupakan nilai feromon yang terkait (menunjukkan kekuatannya), atau dalam kata lain merupakan jejak atau penanda yang disisakan dalam *route* terbaik yang dilalui. Semakin pendek jalur yang dilalui maka semakin efisien proses transmisi data tersebut. Dalam konsep ACO, makin sering jalur terpendek tersebut dilalui maka semakin tinggi tingkat probabilitasnya dan data akan secara otomatis akan dikirimkan secara berulang terus menerus pada *route* tersebut.

Sementara itu, untuk menginisiasi tingkat feromon pada semua edge digunakan fungsi '**pheromone = ones(dataset.nodeNo, dataset.nodeNo)**'. Selain itu, metode ACO yang diterapkan pada percobaan tersebut menggunakan 5 parameter, yaitu **Alpha (1)** untuk mengontrol pengaruh tingkat feromon pada penentuan *route*, jika nilai alpha lebih tinggi akan membuat semut lebih bergantung pada jejak feromone yang ada saat memilih jalur berikutnya. Kedua, parameter **Beta (2)** untuk mengontrol visibilitas pada penentuan *route*. Ketiga, parameter **Rho (0.5)** untuk menentukan tingkat

evaporasi feromon dalam mencegah konvergensi. Keempat, parameter ***Q(100)*** untuk menentukan seberapa banyak feromon yang ditambahkan berdasarkan kualitas jalur. Kelima, parameter ***numAnts & maxIterations*** untuk menentukan jumlah semut yang terlibat di dalam setiap iterasi dan jumlah iterasi yang dijalankan.

Proses iterasi yang dilakukan oleh ACO melibatkan konsep visibilitas, dimana visibilitas tersebut dihitung sebagai kebalikan dari jarak '***visibility = 1 ./ (dataset.euclidian + eye(dataset.nodeNo))***'. Kemudian feromon diperbarui dengan mengurangi berdasarkan tingkat evaporasi '***pheromone = (1 - rho) * pheromone***' dan menambahkan berdasarkan sepanjang jalur yang ditemukan oleh semut '***pheromone(path(j), path(j+1)) = pheromone(path(j), path(j+1)) + Q / lengths(k)***'.

Konsep ACO dalam transmisi data cukup efisien, hanya saja dalam beberapa kasus membutuhkan iterasi yang banyak dalam menentukan jalur terbaik pengiriman data, sehingga akan memakan waktu yang lama dan membutuhkan energi yang tinggi hingga data dikirim ke *node* tujuan. Oleh karena itu, ACO lebih cocok diimplementasikan pada konsep jaringan sensor nirkabel dengan pengiriman data yang tidak terlalu banyak.

3.2. Shortest Path

Pada percobaan yang telah dilakukan dapat dilihat bagaimana algoritma *shortest path* bekerja, dimana dari rangkaian *node* yang saling terhubung, *node* akan mati satu persatu di setiap iterasinya hingga menemukan *route* terpendek untuk mentransmisikan data. Adapun *route* terpendek tersebut dicari dengan memanfaatkan konsep *graph*, seperti pada kode program dengan fungsi '***shortestpathtree***'

Pada grafik *output* yang dihasilkan oleh algoritma *shortest path* menunjukkan perbandingan total energi dibandingkan dengan jumlah iterasinya. Grafik tersebut menunjukkan penggunaan energi total yang besar pada iterasi awal dan kemudian menurun, dengan beberapa fluktuasi pada iterasi selanjutnya. Berikut adalah beberapa alasan yang mungkin menjelaskan pola tersebut :

1. Penggunaan Energi awal yang tinggi

Pada awal simulasi, banyak *node* yang masih aktif dan terlibat dalam proses *routing*. Karena itu, penggunaan energi total sangat tinggi karena banyak *node* yang mengkonsumsi energi untuk mengirim dan menerima paket data.

2. Penurunan Energi Setelah Iterasi Awal

Setelah beberapa iterasi, beberapa *node* mungkin kehabisan energi dan menjadi tidak aktif (*dead nodes*). Ketika setiap *node* ini menjadi tidak aktif, mereka tidak lagi mengkonsumsi energi, yang menyebabkan penurunan penggunaan energi total. Selain itu, dengan semakin sedikitnya *node* yang terlibat dalam *routing*, total penggunaan energi juga berkurang.

3. Peningkatan Penggunaan Energi pada Iterasi 4 ke 5 :

Peningkatan ini mungkin disebabkan oleh beberapa faktor, seperti :

a. *Rute* baru yang ditemukan

Pada iterasi tertentu, *rute* baru yang lebih panjang mungkin ditemukan yang melibatkan lebih banyak *node*. *Node* tambahan ini akan mengkonsumsi energi lebih banyak, menyebabkan peningkatan penggunaan energi total.

b. Fluktuasi dalam Konsumsi Energi

Konsumsi energi oleh setiap *node* bisa dipengaruhi oleh faktor acak seperti proses pengiriman dan penerimaan paket, serta pemulihan energi. Pada iterasi tertentu, proses ini bisa menyebabkan peningkatan sementara dalam penggunaan energi.

Jadi, dapat disimpulkan dari grafik yang ditampilkan tersebut mencerminkan dinamika dari jaringan sensor nirkabel (WSN) di mana penggunaan energi bervariasi berdasarkan aktivitas *node*, penemuan *rute* baru, dan kondisi acak lainnya yang mempengaruhi konsumsi energi oleh setiap *node* dalam jaringan.

3.3. Perbandingan Performa

Pada grafik *output* pemakaian energi VS iterasi antara Algoritma *Shortest Path* dan Algoritma *Ant Colony*, dapat dilihat bahwa SPA menunjukkan pemakaian energi yang cukup fluktuatif di beberapa iterasi awal dan pemakaian energi cenderung menurun drastis, namun stabil pada iterasi akhir. Sedangkan ACO menunjukkan grafik pemakaian energi yang fluktuatif tetapi menurun secara bertahap, karena energi terus menurun maka *node* mencapai titik stabil yang sangat rendah sesuai dengan jumlah iterasi.

Algoritma *Ant Colony* lebih baik dalam hal menemukan solusi mana jalur yang akan efisien digunakan untuk transmisi data jangka panjang, hal ini terbukti dari grafik pada Gambar 2.11 dimana energi yang digunakan menjadi sangat stabil dan rendah

sesuai dengan jumlah iterasi. Sedangkan untuk Algoritma *Shortest Path* menunjukkan performa yang baik dengan cepat mengurangi pemakaian energi, tetapi fluktuasi yang lebih tinggi pada awal proses jika dibandingkan dengan ACO.

Selain itu, ACO memiliki efisiensi energi yang lebih tinggi. Alasannya karena ACO terus memperbarui jalur berdasarkan jejak feromon yang ditinggalkan, sehingga memungkinkan adaptasi yang lebih baik terhadap perubahan kondisi jaringan, serta pemakaian energi pada ACO cenderung menurun secara bertahap dan mencapai stabilitas yang lebih baik daripada SPA.

Jadi, dapat disimpulkan bahwa dari segi efisiensi iterasi, Algoritma *Shortest Path* bersifat lebih efisien untuk menemukan *best path* dibandingkan dengan Algoritma *Ant Colony*, karena Algoritma *Ant Colony* (ACO) membutuhkan banyak iterasi untuk *best path*-nya. Tetapi dari segi efisiensi energi, ACO cenderung lebih unggul dalam konteks efisiensi energi, karena ACO dapat menyesuaikan *route* berdasarkan kondisi yang terus berubah, seperti variasi energi pada *node*. SPA lebih baik dalam konteks pemakaian energi yang cepat menurun dan stabil pada jaringan yang statis atau tidak sering berubah. Namun, pada jaringan yang dinamis, SPA mungkin tidak seefisien ACO karena kurangnya mekanisme adaptasi yang terus-menerus. Jadi, jika efisiensi energi dan adaptabilitas adalah prioritas utama, maka ACO lebih bagus. Namun, jika stabilitas cepat pada jaringan statis yang diutamakan, SPA bisa lebih cocok.

BAB IV

PENUTUP

4.1. Kesimpulan

1. Dari segi efisiensi energi, algoritma *Ant Colony Optimization* (ACO) cenderung lebih efisien dalam penggunaan energi karena mampu menyesuaikan *rute* berdasarkan perubahan kondisi jaringan. Hal ini terlihat dari grafik yang menunjukkan bahwa penggunaan energi oleh ACO menurun secara bertahap dan mencapai stabilitas yang lebih baik dibandingkan dengan SPA.
2. Dari segi stabilitas dan adaptabilitas, algoritma *Ant Colony Optimization* (ACO) memiliki mekanisme pembaruan *rute* yang berkelanjutan melalui jejak feromon, memungkinkan adaptasi yang lebih baik terhadap perubahan kondisi jaringan seperti variasi energi pada *node*. Ini membuat *Ant Colony Optimization* (ACO) lebih cocok untuk jaringan yang dinamis. Sementara itu, SPA cenderung lebih stabil pada jaringan statis tetapi kurang adaptif terhadap perubahan yang sering terjadi.
3. Dari segi kecepatan dalam menemukan jalur optimal, Algoritma *Shortest Path* (SPA) menunjukkan kemampuan untuk dengan cepat mengurangi pemakaian energi dan menemukan jalur terbaik dalam beberapa iterasi awal. Namun, efisiensi ini disertai dengan fluktuasi energi yang tinggi di awal proses. ACO, di sisi lain, membutuhkan lebih banyak iterasi untuk mencapai jalur terbaik tetapi memberikan stabilitas energi yang lebih baik dalam jangka panjang.

4.2. Saran

1. Disarankan setiap anggota kelompok dapat memahami konsep cara kerja dari Algoritma *Ant Colony* dan *Shortest Path* sebelum memulai percobaan pada MATLAB.
2. Disarankan untuk membaca berbagai sumber seperti jurnal-jurnal terkait Algoritma *Ant Colony* dan *Shortest Path* untuk melakukan analisa terhadap keluaran yang didapatkan.
3. Setiap program yang dijalankan baik untuk algoritma SPA maupun ACO, tetap harus diperhatikan apakah program yang dijalankan telah sesuai dengan pemahaman tentang algoritma tersebut.

REFERENSI

- [1] SOCS BINUS, "Ant Colony Optimization," Dec. 31, 2019. [Online]. Available: <https://socs.binus.ac.id/2019/12/31/ant-colony-optimization/>. [Accessed: Jun. 25, 2024].
- [2] D. Yan, L. Wu, Z. Hou, and S. Liu, "An Improved Ant Colony Optimization for Solving a Vehicle Routing Problem," *Measurement*, vol. 74, pp. 189-200, 2015. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0142061515005840>. [Accessed: Jun. 25, 2024].
- [3] L. Taufiq, I. B. Hanafiah, and F. Saad, "Pencarian Jalur Terpendek Menggunakan Algoritma Ant Colony Optimization," *Neliti*, 2019. [Online]. Available: <https://www.neliti.com/id/publications/297878/pencarian-jalur-terpendek-menggunakan-an-algoritma-ant-colony-optimization>. [Accessed: Jun. 25, 2024].
- [4] HackerEarth, "Shortest Path Algorithms," [Online]. Available: <https://www.hackerearth.com/practice/algorithms/graphs/shortest-path-algorithms/tutorial/>. [Accessed: Jun. 27, 2024].
- [5] R. Munir, "Makalah Stima: Algoritma Ant Colony Optimization," *Informatika STEI ITB*, 2016. [Online]. Available: <https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2015-2016/Makalah-2016/MakalahStima-2016-078.pdf>. [Accessed: Jun. 27, 2024].

LAMPIRAN

Lampiran 1. *Source Code Ant Colony*

```
//ACO Code
clear all
close all
clc

%% Main configuration values for this simulation
dataset.nodeNo = 50; % Number of nodes
dataset.nodePosition(1,:) = [1 50 50]; % (Sender node fixed position)
dataset.nodePosition(2,:) = [2 900 900]; % (Receiver node fixed position)
dataset.NeighborsNo = 5;
dataset.range = 250; % Tolerance distance to become neighbor of one node (Euclidean distance based)
dataset.atenuationFactor = 1.8; % Atenuation factor in freespace - ranges from 1.8 to 4 due environment
dataset.minEnergy = 80; % Mw - Milliwatts (70% energy)
dataset.maxEnergy = 100; % Mw - Milliwatts (Full energy (100%) - 1 mAh charge capacity within 1 Volt energy)
dataset.energyconsumptionperCicle = 0.35;
dataset.energyrecoveryperCicle = 0.2;
dataset.energyfactor = 0.001;
STenergy = 10000;
packet = 0;
iterationcounter = 1;

% Node position sortition
for a = 3 : dataset.nodeNo
    dataset.nodeId = a;
    garbage.x = randi([1 900]); % Xpos sortition
    garbage.y = randi([1 900]); % Ypos sortition
    dataset.nodePosition(a,:) = [dataset.nodeId garbage.x garbage.y]; % NodeID, X and Y position into nodePosition table
end

% Euclidean Distance calc from one node to all others
for i = 1 : dataset.nodeNo
    for j = 1: dataset.nodeNo
        garbage.x1 = dataset.nodePosition(i,2);
        garbage.x2 = dataset.nodePosition(j,2);
        garbage.y1 = dataset.nodePosition(i,3);
        garbage.y2 = dataset.nodePosition(j,3);

        dataset.euclidiana(i,j) = sqrt( (garbage.x1 - garbage.x2) ^2 + (garbage.y1 - garbage.y2)^2 );
    end
end

% Edges matrix definition due "range" variable value
dataset.weights = 1t(dataset.euclidiana,dataset.range);

% Graph construction
G = graph(dataset.weights,'omitselfloops'); % Graph creation based on adjacency matrix (Edges matrix) built above

% Euclidean distance extraction for all existing end-to-end formed by
% "distance tolerance" (range variable value)
for a = 1 : height(G.Edges)
```



```

garbage.s = G.Edges.EndNodes(a,1);
garbage.t = G.Edges.EndNodes(a,2);
garbage.Z(a,:) = dataset.euclidiana(garbage.s,garbage.t);
end
G.Edges.Euclidiana = garbage.Z(:,1);

% Initial energy sortition (from 70% to 100% - minEnergy and maxEnergy variable value)
[dataset.nodePosition(:,4)] = dataset.maxEnergy
-(dataset.maxEnergy-dataset.minEnergy)*rand(dataset.nodeNo,1);
dataset.nodePosition(1:2,4) = STenergy;

% All "G" (Graph object) based nodes degree to use as "node processing
% status overload" (more connections, busier!)
for a = 1: length(dataset.nodePosition(:,1))
    dataset.nodePosition(a,5) = degree(G,dataset.nodePosition(a,1));
end

% Pathloss calc of each Edges based in a freespace (1.8 factor)
[G.Edges.Pathloss] = (10*dataset.attenuationFactor)*log10(G.Edges.Euclidiana);

% End points coordinates and energy migration to G object
for a = 1 : height(G.Edges)
    garbage.Sourcenode = G.Edges.EndNodes(a,1);
    garbage.Targetnode = G.Edges.EndNodes(a,2);
    G.Edges.SourcenodeXpos(a) = dataset.nodePosition(garbage.Sourcenode,2);
    G.Edges.SourcenodeYpos(a) = dataset.nodePosition(garbage.Sourcenode,3);
    G.Edges.TargetnodeXpos(a) = dataset.nodePosition(garbage.Targetnode,2);
    G.Edges.TargetnodeYpos(a) = dataset.nodePosition(garbage.Targetnode,3);
    G.Edges.ActiveEdge(a) = 1;
end

% Graph objects plot
figure('units','normalized','innerposition',[0 0 1 1],'MenuBar','none')
subplot(1,1,1) % 1,3,1 (Line number, column number, graph id) - if you want to show more than 1 graph in
the same window
garbage.Xmax = 1500;
garbage.Xmin = 0;
garbage.Ymax = 1500;
garbage.Ymin = 0;
p = plot(G,'XData',(dataset.nodePosition(:,2)),'YData',(dataset.nodePosition(:,3)));
line(dataset.nodePosition(1:2,2),dataset.nodePosition(1:2,3),'color','green','marker','o','linestyle','none',
'markersize',50)
garbage.ax = gca;
garbage.ax.XAxis.TickValues = 0:100:1000;
garbage.ax.YAxis.TickValues = 0:100:1000;
grid on
hold on
title(['Original WSN | ', 'Nodes number: ', num2str(dataset.nodeNo), ' | Nodes range: ',
num2str(dataset.range)])
pause(2)

% Creation of some stuff needed
garbage.deadodelist = [];
garbage.deadnodeneighbors = [];

%% ACO Parameters
alpha = 1; % Influence of pheromone
beta = 2; % Influence of heuristic information (visibility)
rho = 0.5; % Pheromone evaporation rate

```

```

Q = 100; % Constant for pheromone update
numAnts = 10; % Number of ants
maxIterations = 100; % Number of iterations

% Initialize pheromone levels
pheromone = ones(dataset.nodeNo, dataset.nodeNo);

% Visibility (inverse of distance)
visibility = 1 ./ (dataset.euclidiana + eye(dataset.nodeNo)); % Add identity to avoid division by zero

% Store best path length per iteration
bestPathLengths = zeros(maxIterations, 1);
globalBestPath = [];
globalBestLength = inf;

% Store total energy usage per iteration
totalEnergyUsage = zeros(maxIterations, 1);

%% ACO Algorithm
for iter = 1:maxIterations
    paths = cell(numAnts, 1);
    lengths = zeros(numAnts, 1);
    iterationEnergyUsage = 0; % Initialize energy usage for this iteration

    for k = 1:numAnts
        path = [1];
        while path(end) ~= 2
            i = path(end);
            probabilities = (pheromone(i,:) .^ alpha) .* (visibility(i,:) .^ beta);
            probabilities(path) = 0; % Prevent revisiting nodes
            probabilities = probabilities / sum(probabilities);
            nextNode = find(rand < cumsum(probabilities), 1);
            path = [path nextNode];
        end
        paths{k} = path;
        lengths(k) = sum(dataset.euclidiana(sub2ind(size(dataset.euclidiana), path(1:end-1), path(2:end))));

        % Calculate energy usage for the path
        for j = 1:length(path)-1
            iterationEnergyUsage = iterationEnergyUsage + dataset.energyconsumptionperCicle;
        end
    end

    % Update pheromone levels
    pheromone = (1 - rho) * pheromone;
    for k = 1:numAnts
        path = paths{k};
        for j = 1:length(path)-1
            pheromone(path(j), path(j+1)) = pheromone(path(j), path(j+1)) + Q / lengths(k);
            pheromone(path(j+1), path(j)) = pheromone(path(j+1), path(j)) + Q / lengths(k); % Undirected
        end
    end

    % Find the best path of this iteration
    [minLength, bestAnt] = min(lengths);
    bestPath = paths{bestAnt};

```

```

    % Update global best path
    if minLength < globalBestLength
        globalBestLength = minLength;
        globalBestPath = bestPath;
    end

    % Store the best path length
    bestPathLengths(iter) = globalBestLength;

    % Store total energy usage for this iteration
    totalEnergyUsage(iter) = iterationEnergyUsage;

    % Plotting for each iteration (optional)
    figure('units','normalized','innerposition',[0 0 1 1],'MenuBar','none')
    p = plot(G,'XData',(dataset.nodePosition(:,2)),'YData',(dataset.nodePosition(:,3)));

    line(dataset.nodePosition(1:2,2),dataset.nodePosition(1:2,3),'color','green','marker','o','linestyle','none',
    'markersize',50)
    garbage.ax = gca;
    garbage.ax.XAxis.TickValues = 0:100:1000;
    garbage.ax.YAxis.TickValues = 0:100:1000;
    hold on

    % Plot the global best path
    for j = 1:length(globalBestPath)-1
        plot([dataset.nodePosition(globalBestPath(j),2) dataset.nodePosition(globalBestPath(j+1),2)], ...
            [dataset.nodePosition(globalBestPath(j),3) dataset.nodePosition(globalBestPath(j+1),3)], 'r',
    'LineWidth', 2);
    end

    title(['ACO Iteration: ',num2str(iter),' | Best Path Length: ',num2str(globalBestLength)])
    grid on
    pause(0.5)
end

% Plot the final global best path
figure('units','normalized','innerposition',[0 0 1 1],'MenuBar','none')
p = plot(G,'XData',(dataset.nodePosition(:,2)),'YData',(dataset.nodePosition(:,3)));
line(dataset.nodePosition(1:2,2),dataset.nodePosition(1:2,3),'color','green','marker','o','linestyle','none',
'markersize',50)
garbage.ax = gca;
garbage.ax.XAxis.TickValues = 0:100:1000;
garbage.ax.YAxis.TickValues = 0:100:1000;
hold on

% Plot the global best path
for j = 1:length(globalBestPath)-1
    plot([dataset.nodePosition(globalBestPath(j),2) dataset.nodePosition(globalBestPath(j+1),2)], ...
        [dataset.nodePosition(globalBestPath(j),3) dataset.nodePosition(globalBestPath(j+1),3)], 'r',
    'LineWidth', 2);
end

title(['ACO Global Best Path | Length: ',num2str(globalBestLength)])
grid on
pause(0.5)

% Plot best path length vs iterations
figure
plot(1:maxIterations, bestPathLengths, '-o');

```

```

xlabel('Iteration')
ylabel('Best Path Length')
title('ACO Best Path Length vs Iteration')
grid on

% Plot total energy usage vs iterations
figure
plot(1:maxIterations, totalEnergyUsage, '-o');
xlabel('Iteration')
ylabel('Total Energy Usage')
title('Total Energy Usage vs Iteration')
grid on

disp(['ACO Global Best Path Length: ', num2str(globalBestLength)])
disp(['ACO Global Best Path: ', num2str(globalBestPath)])

```

Lampiran 2. Source Code Shortest Path

```

//SPA Code
clear all
close all
clc

%% Main configuration values for this simulation
dataset.nodeNo = 50; % Number of nodes
dataset.nodePosition(1,:) = [1 50 50]; % Sender node fixed position
dataset.nodePosition(2,:) = [2 900 900]; % Receiver node fixed position
dataset.NeighborsNo = 5;
dataset.range = 250; % Tolerance distance to become a neighbor of one node (Euclidean distance based)
dataset.attenuationFactor = 1.8; % Attenuation factor in freespace - ranges from 1.8 to 4 due to environment
dataset.minEnergy = 80; % Mw - Milliwatts (70% energy)
dataset.maxEnergy = 100; % Mw - Milliwatts (Full energy (100%) - 1 mAh charge capacity within 1 Volt energy)
dataset.energyconsumptionperCicle = 0.35;
dataset.energyrecoveryperCicle = 0.2;
dataset.energyfactor = 0.001;
STenergy = 10000;
packet = 0;
iterationcounter = 1;

% Node position sortition
for a = 3 : dataset.nodeNo
    dataset.nodeId = a;
    garbage.x = randi([1 900]); % Xpos sortition
    garbage.y = randi([1 900]); % Ypos sortition
    dataset.nodePosition(a,:) = [dataset.nodeId garbage.x garbage.y]; % NodeID, X and Y position into nodePosition table
end

% Euclidean Distance calculation from one node to all others
for i = 1 : dataset.nodeNo
    for j = 1: dataset.nodeNo
        garbage.x1 = dataset.nodePosition(i,2);
        garbage.x2 = dataset.nodePosition(j,2);
        garbage.y1 = dataset.nodePosition(i,3);
        garbage.y2 = dataset.nodePosition(j,3);
    end
end

```

```

        dataset.euclidiana(i,j) = sqrt((garbage.x1 - garbage.x2)^2 + (garbage.y1 - garbage.y2)^2);
    end
end

% Edges matrix definition due to "range" variable value
dataset.weights = lt(dataset.euclidiana,dataset.range);

% Graph construction
G = graph(dataset.weights,'omitselfloops'); % Graph creation based on adjacency matrix (Edges matrix) built
above

% Euclidean distance extraction for all existing end-to-end formed by "distance tolerance" (range variable
value)
for a = 1 : height(G.Edges)
    garbage.s = G.Edges.EndNodes(a,1);
    garbage.t = G.Edges.EndNodes(a,2);
    garbage.Z(a,:) = dataset.euclidiana(garbage.s,garbage.t);
end
G.Edges.Euclidiana = garbage.Z(:,1);

% Initial energy sortition (from 70% to 100% - minEnergy and maxEnergy variable value)
dataset.nodePosition(:,4) = dataset.maxEnergy - (dataset.maxEnergy - dataset.minEnergy) *
rand(dataset.nodeNo,1);
dataset.nodePosition(1:2,4) = STenergy;

% All "G" (Graph object) based nodes degree to use as "node processing status overload" (more connections,
busier!)
for a = 1: length(dataset.nodePosition(:,1))
    dataset.nodePosition(a,5) = degree(G,dataset.nodePosition(a,1));
end

% Pathloss calculation of each Edges based in a freespace (1.8 factor)
G.Edges.Pathloss = (10 * dataset.attenuationFactor) * log10(G.Edges.Euclidiana);

% End points coordinates and energy migration to G object
for a = 1 : height(G.Edges)
    garbage.Sourcenode = G.Edges.EndNodes(a,1);
    garbage.Targetnode = G.Edges.EndNodes(a,2);
    G.Edges.SourcenodeXpos(a) = dataset.nodePosition(garbage.Sourcenode,2);
    G.Edges.SourcenodeYpos(a) = dataset.nodePosition(garbage.Sourcenode,3);
    G.Edges.TargetnodeXpos(a) = dataset.nodePosition(garbage.Targetnode,2);
    G.Edges.TargetnodeYpos(a) = dataset.nodePosition(garbage.Targetnode,3);
    G.Edges.ActiveEdge(a) = 1;
end

% Graph objects plot
figure('units','normalized','innerposition',[0 0 1 1],'MenuBar','none')
subplot(1,1,1)
garbage.Xmax = 1500;
garbage.Xmin = 0;
garbage.Ymax = 1500;
garbage.Ymin = 0;
p = plot(G,'XData',(dataset.nodePosition(:,2)),'YData',(dataset.nodePosition(:,3)));
line(dataset.nodePosition(1:2,2),dataset.nodePosition(1:2,3),'color','green','marker','o','linestyle','none
','markersize',50)
garbage.ax = gca;
garbage.ax.XAxis.TickValues = 0:100:1000;
garbage.ax.YAxis.TickValues = 0:100:1000;
grid on

```

```

hold on
title(['Original WSN | ', 'Nodes number: ', num2str(dataset.nodeNo), ' | Nodes range: ',
num2str(dataset.range)])
pause(2)

% Creation of some stuff needed
garbage.deadodelist = [];
garbage.deadnodeneighbors = [];

%% Finding routes
% Finding shortest path route
G2 = shortestpathtree(G,1,2);

% Initialization of data collection for energy plotting
total_energy_usage = [];

%% Initialize patch existence test for loops
fileID = fopen('report-noACO-simulation.txt','w'); % Open file to collect data to external file
fprintf(fileID,'%6s %20s %20s %20s %20s\r\n','|NodeNo|','|No ACO Scene|','|Hops|','|Packets sent|','|Dead
node|');

% Initialize hopsnumber to avoid errors
hopsnumber = 0;

while ~isempty(G2.Edges)
    G2 = shortestpathtree(G,1,2);
    iterationcounter = iterationcounter + 1;
    % Test if there is connection between node 1 and 2. If not, terminate!
    if isempty(G2.Edges)
        break
    end

    % Find edges found by shortestpathtree in main G object
    for a = 1 : height(G2.Edges)
        source = G2.Edges.EndNodes(a,1);
        target = G2.Edges.EndNodes(a,2);
        garbage.edgesrow(a,:) = findedge(G,source,target);
    end

    % Find nodes involved in routing event
    garbage.routingnodes = unique(G2.Edges.EndNodes);
    garbage.routepath = shortestpath(G,1,2);

    % Construct localization dataset to nodes involved in routing event for plot effects
    for a = 1 : length(garbage.routingnodes)
        garbage.b = garbage.routingnodes(a,1);
        dataset.routingnodesPosition(a,:) = dataset.nodePosition(garbage.b,:);
    end

    % Find nodes not involved in routing to energy increase
    [garbage.Outroutenodes, garbage.Outroutenodesidx] =
setdiff(dataset.nodePosition(:,1),garbage.routingnodes(:,1),'stable');

    % Code block for energy usage (decrease) and packet send count
    total_energy_before = sum(dataset.nodePosition(:,4), 'omitnan');
    while min(dataset.nodePosition(:,4)) > 0
        for a = 1 : length(garbage.routingnodes)
            node = garbage.routingnodes(a,1);

```

```

dataset.nodePosition(node,4) = dataset.nodePosition(node,4) -
dataset.energyconsumptionperCicle^rand() + dataset.energyrecoveryperCicle^rand();
    packet = packet + 1;
end
end
total_energy_after = sum(dataset.nodePosition(:,4), 'omitnan');
total_energy_usage = [total_energy_usage; iterationcounter, total_energy_before - total_energy_after];

% Find dead node ID to discover its neighbors and disable relative edges
[garbage.deadnoderow] = find(dataset.nodePosition(:,4) <= 0);
for a = 1 : length(garbage.deadnoderow)
    deadnode = garbage.deaddatasetnoderow(a,1);
    for b = 1 : height(G.Edges)
        if ismember(G.Edges.EndNodes(b,1),deadnode) == 1 || ismember(G.Edges.EndNodes(b,2),deadnode) ==
1
            G.Edges.ActiveEdge(b) = 0;
        end
    end
end
garbage.deadnodelist(length(garbage.deaddatasetnoderow)+1,1) = deadnode;
[garbage.deadedgerow] = find(G.Edges.ActiveEdge == 0);

% Mark node energy as NaN
[garbage.deaddatasetnoderow] = find(dataset.nodePosition(:,4) <= 0);
for a = 1 : length(garbage.deaddatasetnoderow)
    b = garbage.deaddatasetnoderow(a,1);
    dataset.nodePosition(b,4) = NaN;
end

% Get number of hops between source and destination for this routing session
hopsnumber = length(garbage.routingnodes);

% Informative section in command line
msg = ['Without ACO - Scene #: ',num2str(iterationcounter),' | Hops : ',num2str(hopsnumber),' | Packets
sent: ', num2str(packet),' | Dead node: ', num2str(deadnode),' | Routing nodes: ',
num2str(garbage.routepath)];
disp(msg)

% Plot for every dead edges iteration's result
figure('units','normalized','innerposition',[0 0 1 1],'MenuBar','none')
p = plot(G,'XData',(dataset.nodePosition(:,2)),'YData',(dataset.nodePosition(:,3)));

line(dataset.nodePosition(1:2,2),dataset.nodePosition(1:2,3),'color','green','marker','o','linestyle','none
','markersize',50)
garbage.ax = gca;
garbage.ax.XAxis.TickValues = 0:100:1000;
garbage.ax.YAxis.TickValues = 0:100:1000;
hold on

% Plot all dead nodes in red
for a = 1 : length(garbage.deaddatasetnoderow)
    garbage.b = garbage.deaddatasetnoderow(a,1);

scatter(dataset.nodePosition(garbage.b,2),dataset.nodePosition(garbage.b,3),'MarkerFaceColor','red');
end

% Title for every iteration's result
title(['WSN shortest path: ',num2str(iterationcounter),' |hops: ',num2str(hopsnumber),' |Packets sent:
',num2str(packet),' |Dead node: ', num2str(deadnode),' |Router nodes: ', num2str(garbage.routepath)])

```

```

grid on
pause(2)

% Collect data to append to an external file for later usage
report1 = [dataset.nodeNo; iterationcounter; hopsnumber; packet; deadnode];
fprintf(fileID,'%6.0f %20.0f %20.0f %20.0f %20.0f\r\n',report1);

% Remove dead edges from graph
G = rmedge(G,garbage.deadedgerow(:,1));

% Mark nodes involved in route with green color
scatter(dataset.routingnodesPosition(:,2),dataset.routingnodesPosition(:,3),'MarkerFaceColor','green');
pause(0.2)

% Clear router nodes position to avoid erroneous plot
clear dataset.routingnodesPosition
end

fclose(fileID); % Close external data collector file

% Plot all dead nodes in red
for a = 1 : length(garbage.deadodelist)
    garbage.b = garbage.deadodelist(a,1);
    scatter(dataset.nodePosition(garbage.b,2),dataset.nodePosition(garbage.b,3),'MarkerFaceColor','red');
end

% Title for the last plot
title(['WSN shortest path: ',num2str(iterationcounter),' |hops: ',num2str(hopsnumber),' |Packets sent: ',num2str(packet),' |Dead node: ', num2str(deadnode),' |Router nodes: ', num2str(garbage.routepath),'{\color{red} - ALL ROUTES UNAVAILABLE}'])

% Message if there is no path between source and target even in the first iteration
disp('NO ROUTES BETWEEN SOURCE (NODE1) AND TARGET (NODE2)')

% Plotting total energy usage vs. iteration
figure;
plot(total_energy_usage(:,1), total_energy_usage(:,2));
xlabel('Iteration');
ylabel('Total Energy Usage');
title('Total Energy Usage vs. Iteration');
grid on;
clear all
close all
clc

%% Main configuration values for this simulation
dataset.nodeNo = 50; % Number of nodes
dataset.nodePosition(1,:) = [1 50 50]; % Sender node fixed position
dataset.nodePosition(2,:) = [2 900 900]; % Receiver node fixed position
dataset.NeighborsNo = 5;
dataset.range = 250; % Tolerance distance to become a neighbor of one node (Euclidean distance based)
dataset.attenuationFactor = 1.8; % Attenuation factor in freespace - ranges from 1.8 to 4 due to environment
dataset.minEnergy = 80; % Mw - Milliwatts (70% energy)
dataset.maxEnergy = 100; % Mw - Milliwatts (Full energy (100%) - 1 mAh charge capacity within 1 Volt energy)
dataset.energyconsumptionperCicle = 0.35;
dataset.energyrecoveryperCicle = 0.2;
dataset.energyfactor = 0.001;
STenergy = 10000;

```



```

packet = 0;
iterationcounter = 1;

% Node position sortition
for a = 3 : dataset.nodeNo
    dataset.nodeId = a;
    garbage.x = randi([1 900]); % Xpos sortition
    garbage.y = randi([1 900]); % Ypos sortition
    dataset.nodePosition(a,:) = [dataset.nodeId garbage.x garbage.y]; % NodeID, X and Y position into
    nodePosition table
end

% Euclidean Distance calculation from one node to all others
for i = 1 : dataset.nodeNo
    for j = 1: dataset.nodeNo
        garbage.x1 = dataset.nodePosition(i,2);
        garbage.x2 = dataset.nodePosition(j,2);
        garbage.y1 = dataset.nodePosition(i,3);
        garbage.y2 = dataset.nodePosition(j,3);
        dataset.euclidiana(i,j) = sqrt((garbage.x1 - garbage.x2)^2 + (garbage.y1 - garbage.y2)^2);
    end
end

% Edges matrix definition due to "range" variable value
dataset.weights = lt(dataset.euclidiana,dataset.range);

% Graph construction
G = graph(dataset.weights,'omitselfloops'); % Graph creation based on adjacency matrix (Edges matrix) built
above

% Euclidean distance extraction for all existing end-to-end formed by "distance tolerance" (range variable
value)
for a = 1 : height(G.Edges)
    garbage.s = G.Edges.EndNodes(a,1);
    garbage.t = G.Edges.EndNodes(a,2);
    garbage.Z(a,:) = dataset.euclidiana(garbage.s,garbage.t);
end
G.Edges.Euclidiana = garbage.Z(:,1);

% Initial energy sortition (from 70% to 100% - minEnergy and maxEnergy variable value)
dataset.nodePosition(:,4) = dataset.maxEnergy - (dataset.maxEnergy - dataset.minEnergy) *
rand(dataset.nodeNo,1);
dataset.nodePosition(1:2,4) = STenergy;

% All "G" (Graph object) based nodes degree to use as "node processing status overload" (more connections,
busier!)
for a = 1: length(dataset.nodePosition(:,1))
    dataset.nodePosition(a,5) = degree(G,dataset.nodePosition(a,1));
end

% Pathloss calculation of each Edges based in a freespace (1.8 factor)
G.Edges.Pathloss = (10 * dataset.attenuationFactor) * log10(G.Edges.Euclidiana);

% End points coordinates and energy migration to G object
for a = 1 : height(G.Edges)
    garbage.Sourcenode = G.Edges.EndNodes(a,1);
    garbage.Targetnode = G.Edges.EndNodes(a,2);
    G.Edges.SourcenodeXpos(a) = dataset.nodePosition(garbage.Sourcenode,2);
    G.Edges.SourcenodeYpos(a) = dataset.nodePosition(garbage.Sourcenode,3);

```

```

        G.Edges.TargetnodeXpos(a) = dataset.nodePosition(garbage.Targetnode,2);
        G.Edges.TargetnodeYpos(a) = dataset.nodePosition(garbage.Targetnode,3);
        G.Edges.ActiveEdge(a) = 1;
    end

% Graph objects plot
figure('units','normalized','innerposition',[0 0 1 1],'MenuBar','none')
subplot(1,1,1)
garbage.Xmax = 1500;
garbage.Xmin = 0;
garbage.Ymax = 1500;
garbage.Ymin = 0;
p = plot(G,'XData',(dataset.nodePosition(:,2)),'YData',(dataset.nodePosition(:,3)));
line(dataset.nodePosition(1:2,2),dataset.nodePosition(1:2,3),'color','green','marker','o','linestyle','none',
'markersize',50)
garbage.ax = gca;
garbage.ax.XAxis.TickValues = 0:100:1000;
garbage.ax.YAxis.TickValues = 0:100:1000;
grid on
hold on
title(['Original WSN | ', 'Nodes number: ', num2str(dataset.nodeNo), ' | Nodes range: ',
num2str(dataset.range)])
pause(2)

% Creation of some stuff needed
garbage.deadodelist = [];
garbage.deadnodeneighbors = [];

%% Finding routes
% Finding shortest path route
G2 = shortestpathtree(G,1,2);

% Initialization of data collection for energy plotting
total_energy_usage = [];

%% Initialize patch existence test for loops
fileID = fopen('report-noACO-simulation.txt','w'); % Open file to collect data to external file
fprintf(fileID,'%6s %20s %20s %20s %20s\r\n','|NodeNo|','|No ACO Scene|','|Hops|','|Packets sent|','|Dead node|');

% Initialize hopsnumber to avoid errors
hopsnumber = 0;

while ~isempty(G2.Edges)
    G2 = shortestpathtree(G,1,2);
    iterationcounter = iterationcounter + 1;
    % Test if there is connection between node 1 and 2. If not, terminate!
    if isempty(G2.Edges)
        break
    end

    % Find edges found by shortestpathtree in main G object
    for a = 1 : height(G2.Edges)
        source = G2.Edges.EndNodes(a,1);
        target = G2.Edges.EndNodes(a,2);
        garbage.edgesrow(a,:) = findedge(G,source,target);
    end

    % Find nodes involved in routing event

```

```

garbage.routingnodes = unique(G2.Edges.EndNodes);
garbage.routepath = shortestpath(G,1,2);

% Construct localization dataset to nodes involved in routing event for plot effects
for a = 1 : length(garbage.routingnodes)
    garbage.b = garbage.routingnodes(a,1);
    dataset.routingnodesPosition(a,:) = dataset.nodePosition(garbage.b,:);
end

% Find nodes not involved in routing to energy increase
[garbage.Outroutenodes, garbage.Outroutenodesidx] =
setdiff(dataset.nodePosition(:,1),garbage.routingnodes(:,1),'stable');

% Code block for energy usage (decrease) and packet send count
total_energy_before = sum(dataset.nodePosition(:,4), 'omitnan');
while min(dataset.nodePosition(:,4)) > 0
    for a = 1 : length(garbage.routingnodes)
        node = garbage.routingnodes(a,1);
        dataset.nodePosition(node,4) = dataset.nodePosition(node,4) -
dataset.energyconsumptionperCicle^rand() + dataset.energyrecoveryperCicle^rand();
        packet = packet + 1;
    end
end
total_energy_after = sum(dataset.nodePosition(:,4), 'omitnan');
total_energy_usage = [total_energy_usage; iterationcounter, total_energy_before - total_energy_after];

% Find dead node ID to discover its neighbors and disable relative edges
[garbage.deadnoderow] = find(dataset.nodePosition(:,4) <= 0);
for a = 1 : length(garbage.deadnoderow)
    deadnode = garbage.deaddatasetnoderow(a,1);
    for b = 1 : height(G.Edges)
        if ismember(G.Edges.EndNodes(b,1),deadnode) == 1 || ismember(G.Edges.EndNodes(b,2),deadnode) ==
1
            G.Edges.ActiveEdge(b) = 0;
        end
    end
end
garbage.deaddatasetnoderow(length(garbage.deaddatasetnoderow)+1,1) = deadnode;
[garbage.deadedgerow] = find(G.Edges.ActiveEdge == 0);

% Mark node energy as NaN
[garbage.deaddatasetnoderow] = find(dataset.nodePosition(:,4) <= 0);
for a = 1 : length(garbage.deaddatasetnoderow)
    b = garbage.deaddatasetnoderow(a,1);
    dataset.nodePosition(b,4) = NaN;
end

% Get number of hops between source and destination for this routing session
hopsnumber = length(garbage.routingnodes);

% Informative section in command line
msg = ['Without ACO - Scene #: ',num2str(iterationcounter),' | Hops : ',num2str(hopsnumber),' | Packets
sent: ', num2str(packet),' | Dead node: ', num2str(deadnode),' | Routing nodes: ',
num2str(garbage.routepath)];
disp(msg)

% Plot for every dead edges iteration's result
figure('units','normalized','innerposition',[0 0 1 1],'MenuBar','none')
p = plot(G,'XData',(dataset.nodePosition(:,2)),'YData',(dataset.nodePosition(:,3)));

```

```

line(dataset.nodePosition(1:2,2),dataset.nodePosition(1:2,3),'color','green','marker','o','linestyle','none',
'markersize',50)
    garbage.ax = gca;
    garbage.ax.XAxis.TickValues = 0:100:1000;
    garbage.ax.YAxis.TickValues = 0:100:1000;
    hold on

    % Plot all dead nodes in red
    for a = 1 : length(garbage.deadodelist)
        garbage.b = garbage.deadodelist(a,1);

scatter(dataset.nodePosition(garbage.b,2),dataset.nodePosition(garbage.b,3),'MarkerFaceColor','red');
    end

    % Title for every iteration's result
    title(['WSN shortest path: ',num2str(iterationcounter),' |hops: ',num2str(hopsnumber),' |Packets sent: ',num2str(packet),' |Dead node: ', num2str(deadnode),' |Router nodes: ', num2str(garbage.routepath)])
    grid on
    pause(2)

    % Collect data to append to an external file for later usage
    report1 = [dataset.nodeNo; iterationcounter; hopsnumber; packet; deadnode];
    fprintf(fileID,'%6.0f %20.0f %20.0f %20.0f %20.0f\r\n',report1);

    % Remove dead edges from graph
    G = rmedge(G,garbage.deadedgerow(:,1));

    % Mark nodes involved in route with green color
    scatter(dataset.routingnodesPosition(:,2),dataset.routingnodesPosition(:,3),'MarkerFaceColor','green');
    pause(0.2)

    % Clear router nodes position to avoid erroneous plot
    clear dataset.routingnodesPosition
end

fclose(fileID); % Close external data collector file

% Plot all dead nodes in red
for a = 1 : length(garbage.deadodelist)
    garbage.b = garbage.deadodelist(a,1);
    scatter(dataset.nodePosition(garbage.b,2),dataset.nodePosition(garbage.b,3),'MarkerFaceColor','red');
end

% Title for the last plot
title(['WSN shortest path: ',num2str(iterationcounter),' |hops: ',num2str(hopsnumber),' |Packets sent: ',num2str(packet),' |Dead node: ', num2str(deadnode),' |Router nodes: ', num2str(garbage.routepath),'{\color{red} - ALL ROUTES UNAVAILABLE}'])

% Message if there is no path between source and target even in the first iteration
disp('NO ROUTES BETWEEN SOURCE (NODE1) AND TARGET (NODE2)')

% Plotting total energy usage vs. iteration
figure;
plot(total_energy_usage(:,1), total_energy_usage(:,2));
xlabel('Iteration');
ylabel('Total Energy Usage');
title('Total Energy Usage vs. Iteration');
grid on;

```